AAA Digital Platform — Working with Claude Code

Version 1.0 · 26 July 2026 · Confidential — Ambitious about Autism

A practical guide to changing and improving this platform using Claude Code, written for someone who has never worked with an AI coding assistant before. No prior experience with AI tools is assumed. Some familiarity with using a computer confidently is — you will be typing commands into a terminal.

This document is regenerated from markdown via `npm run handover:build` — see `docs/handover/README.md` for the build process.

> **Read §4 before you do anything else.** It is one habit, it takes ten seconds, and it is the difference between an assistant that changes the right thing and one that confidently changes the wrong thing.

1. What Claude Code Actually Is

Claude Code is an AI assistant that works inside your project's files. You describe what you want in ordinary English; it reads the code, writes changes, runs commands, and tells you what it did.

It is worth being clear about what that does and does not mean.

What it genuinely does well:

- Reads and explains code you don't understand
- Makes changes that span many files at once, consistently
- Finds where something lives in a codebase of thousands of files
- Writes tests, catches its own mistakes when asked to check
- Handles the tedious parts — renaming, updating documentation, tidying

Where it needs you:

- It cannot see your intent, only your words. Vague requests get plausible-looking guesses.
- It can be confidently wrong. It will state something as fact that isn't. You are the check on this.
- It does not know what it hasn't read. This is the single biggest source of bad output, and §4 is entirely about it.

- It cannot make product decisions. "Should learners be able to edit this?" is your call, not its.

Think of it as a fast, tireless, well-read colleague who has just joined and has not yet been shown around. Extremely capable. Needs context. Will not be offended when you correct them.

A note on trust

You do not need to understand every line it writes. You do need to be able to answer: *did the thing I asked for actually happen, and did anything else break?* §8 shows you how to check that without reading code.

2. Before You Start

You need four things. Ask whoever handed over the platform if any are missing.

What	Why	Where
The code on your computer	Claude Code works on local files	`git clone` from GitHub
Node.js installed	Runs the site and its checks	`nodejs.org` — the LTS version
Claude Code installed	The assistant itself	`claude.com/claude-code`
A Claude account	Signs you in	`claude.ai`

Once installed, open a terminal, move into the project folder, and start it:

```
cd ~/Developer/asd-training-app
```

```
claude
```

That's it. You are now in a conversation. Type in plain English and press Enter. Type `/exit` when you're done.

The first thing to run

Before your first real request, install the project's dependencies:

```
npm install
```

This takes a few minutes the first time and only needs doing again when someone changes the project's dependencies.

3. The Project Already Explains Itself

There is a file in the root of this project called `CLAUDE.md`. Claude Code reads it automatically at the start of every session.

It describes how this platform is built — the four user roles, how permissions work, where the training content lives, the database structure, the deployment setup, and a long list of gotchas that have caught people out before. You do not need to read it yourself, and you do not need to explain any of it in your requests. It is already there.

This is why you can ask "add a field to the job openings form" without explaining what job openings are.

Two things follow from this:

1. **If the assistant gets something structurally wrong about this project repeatedly, `CLAUDE.md` may be out of date.** Ask it to check and correct the file. That fixes the problem for every future session, not just this one.
2. **When you learn something the hard way, ask for it to be written down there.** "Add a note to CLAUDE.md that we never push schema changes to production without pushing to dev first." Next session, it knows.

4. The One Habit That Matters Most

Ask it to read the relevant files before it changes anything.

Claude Code does not automatically know what is in every file. When you ask for a change, it decides what to read based on your request. If your request is vague, it may read the wrong things, or too few things, and write code based on an assumption rather than on what is actually there.

The fix is to say so explicitly. Compare:

> ❌ "Add a phone number field to organisations."

> ✅ "Read the Organisation model in `prisma/schema.prisma` and the organisation form components first, then add a phone number field. Show me what you found before changing anything."

The second version produces a change that fits the existing patterns — the same validation style, the same field ordering, the same place in the form. The first produces something that works in isolation and looks foreign next to everything around it.

You do not need to know which files are relevant. These all work:

- "Read the files involved in X before you start."
- "Have a look at how announcements are done, then do the same for notices."
- "Explore how permissions work in this app, then tell me what you found."

Ask it to explore before it acts

For anything you are unsure about, split it into two steps:

Step one — understanding:

> "Explain how a learner gets access to a training programme. Read whatever you need to and walk me through it. Don't change anything yet."

Step two — acting:

> "Right. Now change it so that X."

This costs you one extra message and saves you from changes built on a misunderstanding. It also teaches you how your own platform works, which compounds.

Ask it to check its own work

Two phrases worth keeping in your pocket:

> "Before you write anything, tell me what you're going to change and why."

> "Check that against the actual code — don't rely on what you remember."

The second is surprisingly effective. It genuinely does go back and verify, and it does sometimes come back with "you're right, I was wrong about that."

5. How to Phrase a Request

A good request has three parts. Not all are needed every time, but the more you include, the better the result.

Part	Example
What you want	"Add a 'Closed' filter to the job openings list"
Where, roughly	"on the org admin side, at /admin/jobs"
What good looks like	"match how the status filter works on the announcements page"

Put together:

> "Add a 'Closed' filter to the job openings list on the org admin side at /admin/jobs. Read how the status filter on the announcements page works first and match that pattern."

Useful framings

When you don't know what's possible:

> "Is there a way to let org admins see which of their learners haven't started their training yet? Tell me what would be involved before doing anything."

When something is broken:

> "When I click Save on the workshop form nothing happens. Find out why."

Do not guess at the cause. Describe what you did and what happened. Diagnosing is what it is good at.

When you want an opinion:

> "We're thinking of letting schools write their own home page content. What would that involve, and what could go wrong?"

When you want it to stop and think:

> "Don't write any code yet. Plan this out and show me the plan first."

For anything touching more than a couple of files, this is worth doing every time.

Things that make results worse

- **Several unrelated requests in one message.** Do them one at a time.
- **"Fix everything wrong with this page."** Too vague to act on well. Ask what's wrong first, then pick.
- **Assuming it remembers a previous conversation.** Each new session starts fresh, apart from `CLAUDE.md` and any saved notes. Re-state what matters.
- **Accepting "done" without checking.** See §8.

6. Working in Small Steps

The strongest predictor of a good outcome is small, checkable steps.

A change like "add a document upload to job openings for org admins" is really four changes: the database, the upload endpoint, the form, and the permission check. Asked as one instruction, you get all four at once and no easy way to tell which part is wrong if something misbehaves.

Asked as four, you can confirm each before moving on — and if step three goes sideways, steps one and two are still good.

You do not have to work out the steps yourself:

> "Break this into steps and show me the list. We'll do them one at a time."

7. Your Safety Net: Git

Git records every version of the project. It is what lets you try things without fear.

Two habits protect you completely.

Work on a branch, not on `main`

`main` is the live version. Before starting anything, ask:

> "Create a branch for this work."

Now nothing you do can affect the live site until you deliberately merge it.

Commit whenever something works

A commit is a save point you can return to.

> "Commit this."

If the next change makes things worse:

> "Undo the last change and take us back to the last commit."

You cannot lose committed work by making a mistake afterwards. Commit early and often — every time something works, even partially. This one habit removes most of the risk from experimenting.

Seeing what changed

> "What have you changed so far?"

Plain-English summary, no git knowledge needed.

8. Checking the Work

The assistant will tell you what it did. Trust but verify — and you can verify without reading a line of code.

Ask it to prove it

> "Run the tests and show me the result."

> "Does the site still build?"

The command it runs for the second is:

```
npm run build
```

This regenerates the database client, runs the full automated test suite, and compiles the whole site. If it passes, the change is structurally sound. **If it fails, do not merge, do not deploy.** Paste the error back in and say "this failed, fix it."

Look at it yourself

For anything that changes what people see:

> "Start the dev server so I can look at it."

Then open `http://localhost:3000` in your browser and click around. There is no substitute for this. The tests confirm nothing is broken; only you can confirm it is *right*.

The three questions

After any change, ask yourself:

1. Did the thing I asked for actually happen?
2. Did it change anything I didn't ask about?
3. Does the site still build and pass its tests?

If you are unsure of any of them, ask. "Did this change affect anything other than the job openings page?" is a perfectly good question.

9. Four Worked Examples

Real tasks on this platform, phrased the way that works.

Example 1 — Change some wording

> "The workshops page says 'Virtual Classroom Sessions' as the heading. Change it to just 'Workshops' everywhere it appears for org admins. Find all the places first and show me the list before changing them."

Simple, but note the pattern: **find first, show the list, then change.** Wording often appears in more places than you'd expect.

Example 2 — Add a field

> "I want to record a start date on job openings. Read the JobOpening model in `prisma/schema.prisma` and the job form component, then tell me everything that would need to change. Don't write anything yet."

Review the plan, then:

> "Good. Do it, but leave the database migration until last so we can check the rest first."

Example 3 — Understand something before deciding

> "Explain how a document collection in the library becomes visible to a particular learner. Read whatever you need to. I want to understand it before I decide whether to change it."

Purely a learning request. Entirely legitimate, and the fastest way to get up to speed on a platform you have inherited.

Example 4 — Something is broken

> "An org admin at one of the schools says the Reports page shows no data, but they definitely have learners who've completed training. Investigate — don't fix anything yet, just find out what's happening."

Diagnose first, fix second. Fixing before diagnosing produces changes to code that was never the problem.

10. Where to Be Careful

Most of what you can do is safe and reversible. A few things are not. This is the honest list.

Situation	Why it matters	What to do
Database schema changes	Applied to a live database, they can affect real user data	Always ask "is this reversible?" first. This project has separate dev and production databases — changes go to dev first, always.

Anything touching `.env` files	These hold passwords and API keys	Never paste their contents into a chat, anywhere. Ask it to read the file itself if needed.
Deleting things	Deleted records may not be recoverable	Ask "what exactly will this delete?" before agreeing. Prefer deactivating over deleting — this platform is built for that.
Deploying to production	It's live immediately, for every user	Only after `npm run build` passes and you have looked at it yourself. Never on a Friday afternoon.
Bulk changes to user accounts	Affects real people's access	Ask for the list of who will be affected before it runs.

Two phrases worth memorising:

> "What's the worst case if this goes wrong?"

> "Is this reversible?"

Ask them whenever you feel uncertain. Uncertainty is information — it usually means you have not been told enough yet.

It will sometimes be wrong

It will occasionally state something about this platform with total confidence and be mistaken. This is normal and not a sign anything is broken.

Your defence is not vigilance, it is verification: run the build, look at the page, ask it to check its claim against the actual code. When you catch an error, say so plainly — "that's not right, the reports page is at /admin/reports not /reports" — and it will correct course. There is no need to be diplomatic.

11. When Things Go Wrong

What you see	What it usually means	What to say
It changed more than you expected	Your request was broader than intended	"Undo that. I only wanted the heading changed, nothing else."
The build fails	Something doesn't compile or a test broke	Paste the error. "This failed — here's the error. Fix it."

It's going in circles	It's missing context	"Stop. Read `<file>` and tell me what you actually see there."
It says done but nothing changed	Change went somewhere unexpected	"Show me exactly which files you changed."
You've lost track	Too many changes at once	"Summarise everything you've changed since we started."
It won't do something	Genuine safety limit, or missing permission	Read its explanation — it usually says exactly what it needs.

The universal escape hatch: press `Esc` to interrupt it mid-work. Nothing is broken by stopping. You can then redirect: "Stop — that's not what I meant. What I actually want is…"

12. Getting Better Over Time

Tell it your preferences

> "From now on, always show me a plan before changing more than one file."

> "Remember that I prefer British English spelling in all user-facing text."

These stick across sessions.

Keep `CLAUDE.md` alive

Every time you discover something non-obvious about this platform — a step that must happen in a certain order, a setting that catches people out, a decision and its reason — ask for it to be added to `CLAUDE.md`. It is a shared brain for everyone who works on this platform afterwards, human or otherwise.

Read what it tells you

When it explains why it did something a particular way, that explanation is usually worth reading. Over a few weeks of this you will find you have absorbed a surprising amount about how the platform works — which makes your requests sharper, which makes the results better.

13. Quick Reference

Starting up:

```
cd ~/Developer/asd-training-app
```

```
claude
```

Checking everything still works:

```
npm run build
```

Looking at the site locally:

```
npm run dev
```

Then open `http://localhost:3000`.

Phrases that consistently improve results:

- "Read the relevant files first, then tell me what you found."
- "Don't change anything yet — plan it and show me."
- "Break this into steps. We'll do them one at a time."
- "Check that against the actual code."
- "Run the tests and show me the result."
- "What's the worst case if this goes wrong?"
- "Create a branch for this."
- "Commit this."
- "Undo that."
- "Summarise everything you've changed."

In one line: *make it read before it writes, work in small steps, commit whenever something works, and check the build before you believe anything is finished.*

14. Glossary

Term	What it means
Branch	A parallel copy of the project where you can work without affecting the live version
Build	Compiling and testing the whole site to confirm nothing is broken
Commit	A save point you can return to
Dev server	A copy of the site running on your own computer, for looking at changes
Deploy	Publishing changes so real users see them
Merge	Bringing branch changes into the main version
Migration	A change to the database's structure
Prisma	The tool this project uses to talk to its database
Repository / repo	The project and its full history
Schema	The definition of what data the platform stores
Terminal	The window where you type commands
Test	An automated check that some part of the platform behaves correctly